CHAPTER 4

Error Detection

Hamming distance · parity · 2-D parity · Internet checksum · CRC division · burst-error rules

QUICK TRICK

CRC in 3 moves: (1) append **r zeros** (r = degree of G); (2) divide by G using **XOR**, no borrow; (3) the **remainder** replaces the zeros. Receiver divides again: remainder 0 means "no error detected". Detect d errors needs $d_{min} \geq d + 1$. Correct t errors needs $d_{min} \geq 2t + 1$.

4.1 The idea: add redundancy

● LEVEL 1 · BEGINNER

The problem. Noise flips bits. **The idea.** Add a few extra bits computed from the data. The receiver recomputes them. If they do not match, the frame is damaged. **Analogy.** Writing the total at the bottom of a bill so the reader can check the sums.

Hamming distance between two code words = number of bit positions where they differ. The code distance d_{min} = smallest distance between any two valid code words.

Goal	Needs
Detect up to d bit errors	$d_{min} \geq d + 1$
Correct up to t bit errors	$d_{min} \geq 2t + 1$
Detect d and correct t (d ≥ t)	$d_{min} \geq d + t + 1$

4.2 Parity

● LEVEL 1 · BEGINNER

- **Even parity:** add one bit so the total number of 1s is even. Odd parity: make it odd. Example: data 1011001 has four 1s, so even-parity bit = **0**.
- Detects every **odd** number of bit errors. **Misses any even number** of errors (two flips cancel).
- **2-D parity:** parity for every row and every column. Detects **any 1, 2 or 3** bit errors, and **corrects a single** bit error (row and column point to it). Some 4-bit patterns (a rectangle) are missed.

4.3 Internet checksum

● LEVEL 2 · EXAM

Split the data into 16-bit words. Add them with **end-around carry** (one's complement addition). Take the **complement** of the sum: that is the checksum. Receiver adds all words *including* the checksum: result must be all 1s (0xFFFF). Used in IP, TCP, UDP headers.

```
8-bit demo:  01010101 (0x55) + 11001100 (0xCC) = 1 0010 0001 (0x121)
             wrap the carry: 0x21 + 1 = 0x22 = 00100010
             checksum = complement = 11011101 (0xDD)
Receiver:    0x55 + 0xCC + 0xDD = 0x1FE -> 0xFE + 1 = 0xFF  (all ones, OK)
```

Weak points: it cannot see **re-ordered words** or two errors that cancel. It is cheap, CRC is much stronger.

4.4 CRC (Cyclic Redundancy Check)

● LEVEL 2 · EXAM

Treat the bit string as a polynomial. A generator **G** of degree *r* has *r* + 1 bits. Example: $x^3 + x + 1$ ↔ **1011**. The CRC has **r bits**.

AMJESH (https://singhamjesh.github.io/)

```
Data 1001, G = 1011 (r = 3)   ->  append 3 zeros: 1001000
  1001000
  1011           XOR
  -------
  0010000
    1011         XOR (first 1 is at the third bit)
    -------
  0000110        remainder = 110   ->  transmit 1001 110 = 1001110
```

Check. Receiver divides 1001110 by 1011 and gets remainder 000, so no error is detected. **Classic example (Tanenbaum):** data 1101011111, G = 10011 gives CRC **0010**, frame 11010111110010.

Error type	CRC detects it when
Single-bit error	Always, if G has at least two terms (x^r and 1 present).
Two-bit errors	Always, if G has a factor with at least three terms and frame is short enough.
Odd number of errors	Always, if G is divisible by (x + 1).
Burst of length ≤ r	Always.
Burst of length r + 1	Missed with probability $1/2^{r-1}$.
Longer bursts	Missed with probability $1/2^r$.

Error correction in one box (Hamming code)

Check bits sit at positions 1, 2, 4, 8, … Choose the smallest r with $\mathbf{2^r \geq m + r + 1}$ for m data bits. m = 4 → r = 3 (7,4 code); m = 11 → r = 4 (15,11 code). Syndrome = position of the wrong bit.

4.5 Common mistakes

COMMON MISTAKE

- Appending the wrong number of zeros (it is **r**, the degree, not the length of G).
- Using ordinary subtraction instead of XOR (no borrow, no carry in CRC).
- Forgetting that parity misses an even number of errors.
- Computing the checksum without the end-around carry, or forgetting the final complement.

QUICK TRICK

Memory line: "CRC: append r zeros, XOR-divide, remainder out."

GATE QUICK CHECK — Chapter 4

```
d_min >= d+1 detect d      d_min >= 2t+1 correct t     parity: odd errors only
CRC bits = degree of G     frame sent = data + remainder     good frame -> remainder 0
Internet checksum: add 16-bit words, wrap carry, complement; receiver sum = 0xFFFF
burst <= r always caught   burst r+1 missed 1/2^(r-1)   longer missed 1/2^r
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Even-parity bit for data 1011001.

Q2. Can single parity detect two bit errors?

Q3. Write $x^3 + x + 1$ as a bit string.

Q4. How many CRC bits when G has degree 5?

● LEVEL 2 · EXAM

Q5. Compute the CRC for data 1010001101 with G = 110101.

Q6. Internet checksum of the 16-bit words 0x1234, 0xABCD, 0x00FF.

Q7. Receiver gets 0x1234, 0xABCD, 0x00FF and checksum 0x40FF. What does it do and what does it find?

Q8. Hamming parity bits required for 11 data bits.

● **LEVEL 3 · GATE**

Q9. A code has $d_{min} = 7$. How many errors can it detect and how many correct?

Q10. Smallest d_{min} to detect 3 errors and correct 1 at the same time?

Q11. A 4-bit CRC (r = 4) sees a burst error of exactly 5 bits. Probability it escapes (random pattern)?

Q12. A CRC-16 generator is used. What burst lengths are always detected?

Answers

Q	Solution
1	Four 1s already (even), so the parity bit is **0**.
2	**No.** Two flips keep the parity unchanged.
3	**1011**.
4	**5** bits (append 5 zeros before division).
5	Remainder = **01110**. Transmitted frame: 10100011101**01110**.
6	Sum = 0xBF00 (no carry out). Complement = **0x40FF**.
7	It adds all four words: 0xBF00 + 0x40FF = **0xFFFF**, so no error is detected.
8	$2^r \geq 11 + r + 1 \rightarrow r =$ **4**.
9	Detect up to **6**, correct up to **3** ($2t + 1 \leq 7$).
10	$d + t + 1 = 3 + 1 + 1 =$ **5**. (Check: detect 3 needs 4, correct 1 needs 3, but doing both together needs 5.)
11	$1/2^{r-1} = 1/2^3 =$ **1/8**.
12	All bursts of length **≤ 16** bits.

Next: Chapter 5 — Sliding Window Protocols (stop-and-wait, Go-Back-N, Selective Repeat).

AMJESH singhamjesh.github.io